École Polytechnique Fédérale de Lausanne

# Adaptive Calibration Algorithms for Tunable Analog Peripherals

by Khalil M'hirsi

## Master Thesis

Approved by the Examining Committee:

Prof. Mathieu Salzmann
Thesis Advisor

Richard Lengagne
External Expert

Christophe Constantin
Thesis Supervisor

EPFL CVLAB
BC 309 (Bâtiment BC)
Station 14
CH-1015 Lausanne

7 August 2026

*"I struggled for a long time*
*with surviving.*
*And no matter what,*
*you keep finding something*
*to fight for."*
*— The Last of Us*

# Acknowledgments

I would like to express my sincere gratitude to Christophe Constantin, my supervisor at Logitech, for his trust, guidance, and critical perspective throughout this thesis. I am equally thankful to Yael, a close collaborator on this project, whose feedback and thoughtful challenges helped sharpen both the ideas and the direction of the work. I am also deeply grateful to Professor Mathieu Salzmann for his valuable advice, high standards, and steady guidance in ensuring that this thesis remained rigorous and focused.

I would also like to thank Kilian and Frederic for their technical support on the hardware side, and Jozef and Lilou for the many helpful discussions on machine learning and signal processing. I am grateful as well to my teammates Anais and Kevin, to Thomas, and to the broader team at Logitech for the many forms of support that surrounded this work, from technical discussions and thoughtful advice to the practical organization of data-collection sessions and user tests. This thesis would not have taken its final shape without that collective effort.

On a more personal note, I am deeply grateful to my partner, Sophia, for the patience, steadiness, and care she brought throughout a demanding period. Her presence made the difficult moments lighter and the better ones meaningful. I am equally thankful to my close friends Hugo, Christelle, and Maxence, whose encouragement, listening, and honesty helped me keep moving forward when my own perspective narrowed. More broadly, I am grateful to those closest to me who stood by me through a difficult time and helped me carry this work to its conclusion.

*Lausanne, 7 August 2026* Khalil M'hirsi

## Abstract

Analog peripherals with continuous depth-sensing hardware offer sub-millimetre actuation precision, yet their firmware often relies on static thresholds that ignore per-user biomechanical variance and degrade under neuromotor fatigue. This thesis addresses that gap by designing an end-to-end click-sensitivity recommendation pipeline for Haptic Inductive Trigger System (HITS) devices, using telemetry streamed from the mouse in order to recommend personalized Actuation Point (AP) and Rapid Trigger (RT) settings. The system makes three contributions: (1) a supervised labeling workflow that first generates frame-level labels from algorithmic criteria (depth geometry, speed, and acceleration thresholds) and then iteratively improves those labels through model-assisted refinement; (2) a four-class temporal intent-detection model—supporting Random Forest, Gradient Boosted Trees, and a dilated residual fully-convolutional network—that combines digital signal preprocessing, class-imbalance-aware sample selection, and context-rich features to identify make-onset, click-hold, and break-onset phases from high-frequency telemetry; and (3) a joint CMA-ES optimizer that simulates the firmware click state machine frame-by-frame and finds Pareto-optimal (AP, RT) configurations that reduce actuation latency while maximizing click reliability through explicit constraints on missed clicks and unintended actuations. Together, these components form a practical recommendation workflow from raw HITS telemetry to interpretable parameter recommendations for click sensitivity tuning.

# Contents

# 1. Introduction

Human-Computer Interaction (HCI) in high-performance computing and competitive esports places stringent demands on input peripherals. In environments where users execute several hundred Actions Per Minute (APM), actuation latency and threshold stability can affect both task performance and musculoskeletal load [22, 31].

### 1.1 The Evolution of Continuous Sensing Hardware

Peripheral hardware has shifted from binary mechanical switches toward continuous sensing interfaces. In HCI, threshold selection and activation logic have long been recognized as key determinants of usability and speed [17, 20]. Recent Hall-effect and inductive devices expose depth-dependent actuation and reset behavior to firmware, including Rapid Trigger mechanisms for dynamic re-actuation [6, 26, 36]. Commercial systems such as HITS further combine inductive sensing with configurable haptic feedback [9, 18, 19, 30]. Collectively, these developments increase control resolution but also increase dependence on well-calibrated software thresholds [3, 33, 34].

### 1.2 The Limitations of Static Actuation Thresholds

Despite these hardware leaps, the software interpreting this data remains limited. Current analog peripherals rely on static firmware thresholds that are agnostic to human biomechanical variance. Players exhibit unique control signatures, including micro-hesitations and varying velocity curves, which often clash with rigid hardware logic [17]. When static thresholds are set too aggressively, they fail to filter out physiological noise; when set too conservatively, they introduce unnecessary mechanical pre-travel. This friction results in suboptimal latency or a high frequency of unintended actuations during high-stress scenarios [20, 33].

### 1.3 Neuromotor Fatigue and Performance Decay

These hardware trends also intersect with user health. Higher APM in esports has been associated with repetitive strain injuries and tendinopathies [7, 14, 22]. Biomechanical studies report distinct fatigue patterns for keyboard and mouse interaction [16], and repetitive clicking can degrade aiming accuracy while increasing forearm muscle load [8]. High-APM genres are linked to elevated kinematic demand on wrist extensors [31], with measurable neuromotor adaptation during fatiguing tasks [12]. These findings motivate adaptive calibration that maintains responsiveness while reducing unintended actuation under fatigue.

### 1.4 Toward Adaptive Intent Detection

One longer-term direction is intent inference before full mechanical actuation. Prior work on surface electromyography (sEMG) and embedded TinyML suggests this is technically plausible [2, 15, 28, 35]. However, this thesis focuses on a narrower and directly evaluable scope: offline calibration from depth telemetry already produced by the peripheral. On-device inference is treated as future work (Section 7.3). Productization and UX integration (including the UX Exploration v2 flow and eventual GHub/GHabits integration) are likewise positioned as future work.

### 1.5 Thesis and Contributions

The central hypothesis of this thesis is that per-user variation in click telemetry is structured enough to support automatic labeling, statistical modeling, and parameter optimization beyond static default settings. The contributions are:

- A hybrid labeling pipeline for analog HID click intent that derives initial frame-level `MAKE_ONSET`, `CLICK_HOLD`, and `BREAK_ONSET` annotations from algorithmic criteria (geometry, speed, and acceleration) and then improves boundary quality through model-assisted iteration.
- A four-class temporal sequence classifier with two model paths: a shallow path (Random Forest, Gradient Boosted Trees) operating on handcrafted sliding-window features for sub-second per-session training, and a deep path (dilated residual FCN) operating on raw telemetry with a $\pm 1530$ ms learned receptive field.
- A joint CMA-ES optimizer that simulates the peripheral firmware's Rapid Trigger state machine

and finds Pareto-optimal (AP, RT) configurations, replacing manual threshold tuning with a data-driven trade-off curve.

While not a primary algorithmic contribution, the work also includes a small HCI-facing component: purpose-built minigames that recreate common competitive-gaming input scenarios, providing a structured environment for users to generate calibration telemetry and receive a hardware recommendation. The recommendation outputs are structured for interpretability (trade-off views and comparative candidate points), so users can inspect why a suggested AP/RT point is proposed.

# 2. Background

To understand the design and implementation of an adaptive actuation system, it is necessary to establish the technical foundations of continuous sensing hardware, the mathematical logic of modern trigger parameters, and the biomechanical constraints of the human hand.

## 2.1 Continuous Depth-Sensing Mechanisms

Traditional mechanical switches operate on a binary principle: a circuit is either open or closed based on a physical contact point. In contrast, continuous depth-sensing switches measure the absolute position of the stem throughout its entire travel distance.

**2.1.1 Hall-Effect and Magnetic Sensing.** Hall-effect sensors utilize a permanent magnet attached to the switch stem and a sensor on the Printed Circuit Board (PCB). As the magnet approaches the sensor, the Hall voltage varies proportionally to the magnetic field strength [26, 36]. This allows the firmware to map voltage levels to precise displacement values in millimeters. Compared to classic contact switches, this technology reduces dependence on debounce filtering and enables lower-latency signal processing.

**2.1.2 Haptic Inductive Trigger System (HITS).** Refined in the 2026 Logitech SUPERSTRIKE series, the Haptic Inductive Trigger System (HITS) replaces magnets with an inductive coil architecture [19, 30]. By measuring changes in electromagnetic induction caused by the proximity of a metallic puck within the trigger, HITS provides higher resolution and lower susceptibility to external magnetic interference compared to Hall-effect sensors. Furthermore, HITS integrates localized haptic motors to simulate tactile "bumps" or "clicks" at software-defined depths, decoupling the physical feel of the switch from its electrical actuation point [9].

## 2.2 Control Parameters: AP and RT

The transition to continuous sensing introduced two primary variables that define the "feel" and speed of an input device.

**2.2.1 Actuation Point (AP).** The Actuation Point (AP) is the specific depth ($d$) at which a button-press event is registered. In traditional switches, this is fixed (e.g., 2.0mm). In analog systems, AP is a variable $d_{act} \in [0.1, 4.0]$mm. While these parameters are characteristically expressed as spatial displacement for keyboards, in the context of mice, a specific actuation depth inherently corresponds to a specific physical force applied onto the keyplates. Lowering $d_{act}$ reduces mechanical pre-travel (or actuation force), allowing for faster response times, but increases the risk of accidental actuations caused by resting finger weight.

**2.2.2 Rapid Trigger (RT).** Rapid Trigger (RT) eliminates the fixed reset point found in mechanical switches. In a standard switch, the button must travel back up past a specific reset point before it can be pressed again. RT logic instead monitors the direction of travel. As soon as the stem moves upward by a specific threshold ($\Delta_{up}$), the input resets. Conversely, as soon as it moves back down by $\Delta_{down}$, it reactuates [6, 36]. RT mechanisms differ primarily in their reference boundaries: dynamic RT algorithms track the deepest and highest positions continuously along the entire travel path, only terminating this dynamic tracking when the displacement reaches absolute zero (a complete release). By contrast, other RT implementations restrict this dynamic tracking exclusively to the domain below the Actuation Point (AP), instantly terminating the rapid trigger state if the stem ascends past the initial AP threshold. Both methods allow for rapid repeated clicking, which is critical in high-

APM genres.

### 2.3 Biomechanical Variance and Neuromotor Fatigue

The efficiency of AP and RT is limited by both hardware imperfections and the physiological state of the user. Before examining human motor control, it is important to acknowledge that the raw sensor data is inherently noisy due to analog-to-digital converter (ADC) polling jitter, quantization noise, and non-linearities at the physical extremes of switch travel (top-out and bottom-out crush). However, even assuming a perfectly linear sensor, human motor control is subject to "signal-dependent noise," where the variance of movement increases with the force or speed of the action.

**2.3.1 Resting Pressure and Tremors.** Even at rest, users exert varying degrees of force on a peripheral. Factors such as cognitive stress or caffeine intake can cause micro-tremors-high-frequency, low-amplitude oscillations in displacement telemetry [13]. If the AP is set to 0.1mm and the user's tremor amplitude is 0.15mm, the system will register "ghost" inputs.

**2.3.2 Neuromotor Fatigue Signatures.** Extended sessions lead to fatigue in the *flexor digitorum superficialis* and *extensor digitorum* muscles [8, 16]. Fatigue manifests in two measurable telemetry changes:

1. **Velocity Decay:** The "snap-back" velocity (the speed at which a finger releases a key) decreases as the extensor muscles tire [32].
2. **Resting Weight Shift:** As the flexor muscles lose tonicity, the resting weight of the finger often increases, leading to deeper "resting" displacement in the analog sensor [12, 14].

### 2.4 Constraints of Offline Calibration (Scope Context)

This thesis focuses on offline calibration from depth telemetry. While continuous telemetry at high polling rates could theoretically be processed via on-device inference using quantized INT8 models on peripheral MCUs [15], hardware-in-the-loop deployment requires firmware-level access and strict latency budgets [23]. Consequently, on-device Edge ML is treated as future work, and all calibration pipelines in this study operate offline.

### 2.5 Signal Processing and Optimisation Foundations

The adaptive calibration pipeline draws on several established signal processing and machine learning methods. This section describes their mathematical properties; their problem-specific justifications appear in the Design chapter.

**2.5.1 Savitzky-Golay Smoothing.** Savitzky-Golay (SG) filtering fits a polynomial of degree $p$ to each sliding window of $W$ samples in a least-squares sense and evaluates the polynomial at the window centre [27]. Because the convolution coefficients are precomputed from the normal equations, the runtime cost is $O(n)$ regardless of window width, identical to a moving average. Crucially, the polynomial fit exactly reproduces moments up to degree $p$, so peak positions and edge inflection points in the signal are not distorted—a property absent from symmetric Gaussian or box filters, which both shift extrema toward the window centre.

**2.5.2 Hermite Cubic Interpolation: Akima and Makima.** Akima interpolation constructs a piecewise cubic Hermite spline whose tangent at each knot is a weighted average of the four adjacent finite differences, with weights inversely proportional to the absolute difference between consecutive slopes [1]. This weighting isolates the tangent estimate from outlier slopes: a single anomalous interval does not propagate ringing across the spline, unlike a natural cubic spline whose tangents are coupled globally. Akima does not enforce monotonicity, so it can represent non-monotone smooth curves. Modified Akima (Makima) adds a half-mean-slope regularisation term to the weight formula, preventing zero-weight degeneracy when two adjacent slopes are equal and opposite—a degenerate case that arises frequently in quantized sensor signals.

**2.5.3 Covariance Matrix Adaptation Evolution Strategy.** To optimize physical hardware thresholds like AP and RT, we require black-box optimizers that

handle continuous, correlated variables well. CMA-ES is a derivative-free evolutionary optimiser for continuous search spaces [11]. It maintains a multivariate Gaussian distribution over candidate solutions and, at each generation, moves the distribution mean toward the best-performing candidates and adapts the full covariance matrix to reflect the local curvature of the objective landscape. This covariance adaptation lets the algorithm exploit correlations between parameters (such as the interdependent relationship between optimum AP and RT points) and navigate elongated, ill-conditioned valleys that cause gradient-free methods with diagonal step sizes (e.g. Nelder-Mead or coordinate descent) to stall. CMA-ES converges in $O(n^2)$ function evaluations for an $n$-dimensional problem, far fewer than the $O(N^n)$ evaluations required by grid search over a width-$N$ grid.

**2.5.4 Ensemble Methods: Random Forests and Gradient Boosting.** For the offline ML pipeline, ensemble methods are particularly suited for extracting features from sliding-window time-series data because they are robust to scale differences and irrelevant features. A Random Forest [5] builds an ensemble of decision trees, each trained on a bootstrapped subsample with a random subset of features considered at each split, and averages their posterior class probabilities. The double randomisation decorrelates tree errors, yielding ensemble variance lower than any individual tree without significantly increasing bias. Gradient Boosted Trees [10] instead fit trees sequentially: each tree is trained to correct the pseudo-residual of the current ensemble via gradient descent on a differentiable loss. This staged correction reduces bias relative to a Random Forest, at the cost of longer training time and greater sensitivity to hyperparameter tuning.

**2.5.5 Dilated Fully Convolutional Networks.** While ensemble methods excel with hand-crafted features, deep learning models can automatically extract hierarchical representations directly from raw, unaligned telemetry data. Fully Convolutional Networks (FCNs) adapted for time-series employ stacked layers of 1D dilated convolutions. Dilated convolutions expand the temporal receptive field exponentially without requiring pooling layers that destroy temporal resolution, making them well-suited for capturing long-range dependencies in continuous analog sensor streams [4].

# 3. Related Work

This chapter surveys prior work across the five domains that the proposed system draws on and contributes to: adaptive threshold methods for HID devices, temporal sequence labeling from biosignals, programmatic labeling and weak supervision, dilated convolutional architectures for sequence modeling, and transformer-based event decoding for temporal sequences.

## 3.1 Adaptive Threshold Methods for HID Devices

The problem of optimizing actuation parameters for human input devices has a long history in HCI, though prior approaches have relied on fixed heuristics rather than per-user learned models.

Kim et al. [17] introduced *impact activation*, an alternative actuation scheme for mouse buttons in which a click is registered at the first contact force peak following a rapid finger descent, rather than at a fixed displacement threshold. Their evaluation showed faster reaction times and fewer double-click errors under time pressure. Importantly, impact activation still relies on a single fixed heuristic (the first force peak), making it unsuitable for users with atypical contact dynamics, and it does not adapt to within-session fatigue. This thesis learns thresholds from the user's own recorded telemetry rather than applying a globally fixed policy.

Trewin [33] proposed the *Dynamic Keyboard*, which automatically adjusts keyboard response parameters (minimum press duration, maximum bounce interval) to accommodate users with motor impairments, by observing an initial calibration sequence and adjusting parameters using rule-based adaptation. The Dynamic Keyboard targets binary on/off switch events and adjusts temporal debounce windows; it does not exploit continuous analog depth signals, and its adaptation rules are domain-expert heuristics rather than models trained from user data. The proposed system is complementary in motivation—both aim to reduce

unintended actuations—but operates at a different signal level (continuous depth at 1 kHz vs. binary event timing) and uses a data-driven model rather than hand-coded rules.

To the best of our knowledge, peer-reviewed literature has not yet reported a complete learning-based pipeline for automatic joint tuning of Rapid Trigger (RT) and Actuation Point (AP). Current commercial implementations [18, 36] primarily expose RT and AP as user-adjusted controls without closed-loop personalization. The joint CMA-ES optimizer in this thesis therefore positions RT/AP tuning as an explicitly optimized per-user objective rather than a manual configuration task.

### 3.2 Temporal Sequence Labeling in Biosignal Processing

Detecting click intent from continuous depth telemetry is an instance of temporal sequence labeling, a problem that has received substantial attention in the context of richer biosignals.

Surface electromyography (sEMG) is the closest related modality. Wang et al. [35] presented ReactEMG, which uses masked autoencoder pre-training on sEMG to achieve stable, low-latency intent detection from eight-channel surface electrode recordings. Antuvan and Masia [2] applied Linear Discriminant Analysis to simultaneous classification of wrist and finger movements from sEMG, demonstrating real-time throughput on embedded hardware. Both systems operate on multi-channel electrode signals that encode rich neuromuscular information before mechanical actuation occurs; their signal-to-noise advantage over single-sensor depth telemetry is substantial. The proposed system, by contrast, uses only the depth signal already present in the peripheral itself, requiring no additional sensing hardware. This imposes a harder classification problem—depth telemetry cannot observe pre-actuation motor intent—but removes the deployment barrier of wearable electrodes.

Keystroke dynamics literature [21, 29] models typing patterns for biometric authentication from binary key event timestamps. TypeFormer [29] applies a Transformer architecture to sequences of inter-key timings and achieves high verification accuracy on mobile devices. These methods treat the keystroke as an atomic event and model the temporal pattern between events; they do not model sub-millisecond intra-click dynamics. The proposed system operates at a finer granularity—frame-level classification at 1 kHz—which requires a different feature engineering approach and exposes a different class of failure modes (contact-bounce artefacts, quantization noise) absent from binary event streams.

### 3.3 Weak Supervision and Algorithmic Bootstrapping

The geometric auto-labeler introduced in Section 4.2 fundamentally differs from programmatic weak supervision frameworks such as Snorkel [25]. Snorkel defines *labeling functions*—heuristic rules, distant supervision signals, or domain-expert logic—that each assign noisy labels to unlabeled examples, and combines them via a generative model that learns the accuracy and correlations of each labeling function. The combined label distribution is then used as the training signal for a downstream discriminative model.

Instead of deploying the geometric auto-labeler as an automated weak supervision signal for model training, the proposed system employs it strictly as an algorithmic bootstrapping assistant for human-in-the-loop annotation. Purely algorithmic segments (i.e., those assigned by the auto-labeler but not verified) are explicitly stripped from the dataset during the training pipeline; the machine learning architectures are trained exclusively on user-verified and manually adjusted boundaries. This conservative data policy ensures high-fidelity ground truth and prevents the model from merely mirroring the systematic biases and plateau ambiguities of the underlying geometric heuristic. As a result, the proposed system acts as a strongly supervised learning pipeline accelerated by algorithmic pre-labeling, rather than a weakly supervised system relying on noisy heuristics. To account for natural human inconsistency in selecting the exact millisecond of an onset during this manual process, the system employs asymmetric soft labels (described in Section 4.4) that encode physiologi-

cal directional uncertainty into the verified targets, rather than treating boundary annotations as rigid, perfect temporal observations.

### 3.4 Dilated Convolutions for Sequence Modeling

The OfflineFCN architecture adapts the dilated causal convolution structure introduced by van den Oord et al. in WaveNet [24] for raw audio generation. WaveNet stacks dilated causal convolutional layers with exponentially increasing dilation rates to achieve a large receptive field with manageable parameter counts. The dilation schedule $d_i = 2^i$ allows the stack to cover $\sum 2^i$ steps of context with $O(\log N)$ layers rather than the $O(N)$ layers that a fixed-dilation architecture would require.

The OfflineFCN departs from WaveNet in one critical dimension: it uses symmetric (non-causal) padding rather than causal (left-only) padding. This doubles the effective receptive field per layer for the same kernel size, since context is drawn from both directions—enabling the $\pm 1530$ ms receptive field described in Section 4.4 to be achieved with only 8 blocks. The trade-off is that the resulting model cannot be deployed for streaming inference, since each output depends on future samples. This is appropriate for the offline analysis mode but excludes the model from the on-device path described as future work.

## 4. Design

This chapter details the architecture and algorithmic design of the proposed adaptive actuation system.

### 4.1 System Architecture

The system utilizes a multi-layered software and hardware stack designed to bridge high-frequency physical telemetry with a dynamic optimization engine. At the hardware layer, a modified PRO X2 Superstrike mouse streams continuous 120-level analog depth values at 1 kHz. This bypasses typical firmware constraints that generally limit reporting to mere 4-bit precision at 65 Hz and block reading after haptic events. The hardware communicates with the host operating system via an OS-level driver bridge that interfaces high-level prototyping environments with production-grade HID protocols. Sitting above this is a background sensor service that orchestrates and synchronizes the analog telemetry with OS events and other biometrics. Finally, a front-end orchestration client provides the user dashboard, manages recording state, and drives the downstream machine learning pipelines. The overall component topology is shown in Figure 4.1.

### 4.2 Data Labeling Methodology

Raw analog signals are inherently noisy. Before any label placement, the depth telemetry is smoothed using Modified Akima (MAKIMA) interpolation. MAKIMA was specifically chosen because it preserves signal monotonicity and avoids the severe overshoots characteristic of standard cubic splines, which is critical for accurate boundary detection on a signal with sharp transitions.

**4.2.1 Algorithmic Bootstrapping.** To alleviate the bottleneck of manual annotation at scale, an automated heuristic pipeline pre-populates boundary candidates before human review. Detection anchors on the OS-reported digital key event to identify the approximate click window, then walks the analog signal to locate the true biomechanical boundary:

- **Make Intention:** Starting from the OS make event, the algorithm walks backwards in time to find the first non-zero analog depth, or the immediately preceding local trough or valley in the depth signal. A hysteresis constraint prevents spurious detections from sensor noise near rest position.
- **Break Intention:** Detected at the first zero-crossing of the analog velocity (i.e., the first local peak in depth after the maximum), which typically occurs earlier than the OS-reported key release and more accurately reflects the physical start of the finger lift.

This heuristic correctly handles the vast majority of clean, well-separated clicks, providing an accurate starting point for human reviewers to correct rather than annotate from scratch.

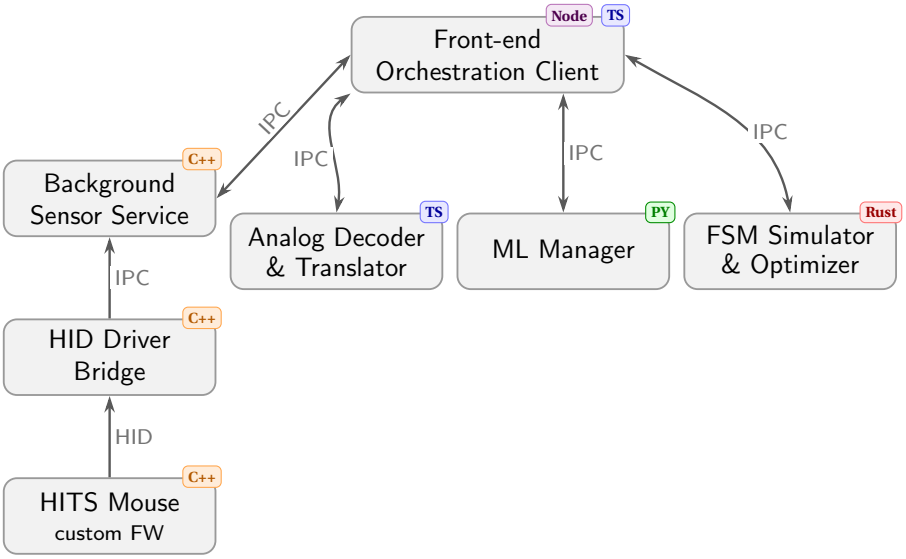


Figure 4.1: System architecture. Telemetry flows upward from a HITS-enabled mouse with custom firmware through a C++ HID driver bridge and a C++ background sensor service via IPC to the front-end orchestration client (Node/Electron + TypeScript). Three independent out-of-process backend services communicate bidirectionally with the client via IPC: an analog decoder and translator (Python + TypeScript), a machine learning manager (Python), and a hardware FSM simulator and optimizer (Rust).

**4.2.2 Iterative Human-in-the-Loop Annotation.** Annotation followed an iterative three-phase strategy designed to maximize label quality while minimizing annotation effort:

1. **Algorithmic bootstrap.** The heuristic pipeline above generated initial boundary candidates for all recordings. Annotators reviewed these within a custom data visualization and labeling interface integrated directly into the front-end orchestration client. The interface renders the raw 1 kHz analog depth signal alongside its derived velocity, and allows annotators to drag start and end boundary markers independently for the left and right button channels. Visual smoothing was applied on a per-recording basis to assist in distinguishing genuine kinetic events from micro-tremors. Annotators focused strictly on biomechanical intent, preserving deliberate presses including those not registered by the OS, while discarding involuntary micro-tremors and resting-weight drift. Approximately 50% of the dataset was verified and corrected in this phase.
2. **Model-assisted labeling.** Once the manually verified corpus was sufficiently large, a preliminary model was trained and used to auto-label the remainder of the dataset. Segments with a model confidence of 95% or above were accepted automatically, covering approximately 60–80% of the remaining unlabeled recordings.
3. **Mixed review.** The remaining recordings — primarily those containing clicks not registered by the OS, which the heuristic cannot anchor on — were processed by a combination of the algorithmic heuristic and direct human annotation. A final manual review pass was conducted over all automatically labeled segments to validate quality before inclusion in the training corpus.

The full extent and breakdown of the resulting annotated corpus is described in Section 5.4.

### 4.3 Data Preparation Pipeline

Raw recordings and their human-verified labels are transformed into normalized arrays shared by all three model architectures. Several design decisions govern this pipeline. First, onset boundary zones are widened by a small tolerance radius to absorb natural annotation imprecision, but are adaptively clamped against adjacent segment boundaries to prevent cross-segment label contamination. Second, derivative channels (velocity, acceleration, jerk, and resting-drift) are derived using zero-phase filtering to avoid introducing causal shift artifacts. Third, idle background periods are trimmed by an activity mask to reduce the dominance of background frames without discarding legitimate negative examples. Fourth, dataset splits are performed at the participant level so no individual's recordings span both training and evaluation. Finally, normalization statistics are fitted on the training partition only, and statistics for kinematic channels are computed over active frames exclusively to prevent background mass from compressing the feature scale. Concrete implementation details—feature extraction windows, normalization constants, and weighting formulas—are described in Section 5.2.

### 4.4 Click-Intent Sequence Modeling

The click-intent problem is formulated as a per-frame four-class temporal segmentation task over continuous 1 kHz sensor streams. Each frame is assigned one of four mutually exclusive labels: Background, Make Onset, Click Hold, and Break Onset. Three model architectures are supported, positioned along a speed–accuracy trade-off curve suited to the realities of on-device, per-session deployment.

**4.4.1 Random Forest.** The Random Forest classifier operates on hand-crafted sliding-window features extracted from a local neighbourhood around each frame. It is the primary production model. Its main strength is training speed: a full session retrains in a few minutes on a consumer CPU. Feature importance scores are directly interpretable, exposing which kinematic cues drive the model's decisions. The main weakness is a bounded temporal horizon limited to a few tens of milliseconds of context, which is sufficient for most click onset detection but may struggle on ambiguous edge cases where the intent signal unfolds over a longer transition. This architecture also fails to capture long-range dependencies and long context of user behavior. The pipeline also requires domain knowledge to design and cannot adapt to latent signal structure that was not anticipated during feature engineering.

**4.4.2 Gradient Boosted Trees.** The Gradient Boosted Trees model shares the same feature extraction pipeline as the Random Forest but replaces the ensemble learner with sequential boosting, which iteratively focuses on the residuals of prior trees. Its key advantage is support for custom loss functions: a focal loss variant down-weights easy background frames, and a Gaussian soft-label objective encodes labeling uncertainty at onset boundaries—both of which reduce over-confidence on the majority background class. Early stopping against a held-out validation metric limits overfitting. The weaknesses are slower training than the Random Forest, greater hyperparameter sensitivity, and the same fundamental constraint of a short-context hand-crafted feature representation.

A distinct design choice for the GBT path is the use of **asymmetric soft labels**. Standard hard labels treat each onset frame as a binary ground truth, but human annotators naturally place boundaries with some imprecision, and the physiological consequences of timing errors are asymmetric. For the Make Onset, detecting slightly *early* is acceptable (the finger is already moving), whereas a *late* detection directly increases perceived input latency. Conversely, for the Break Onset, a slightly *late* detection is tolerable (the click has already been registered), whereas an *early* detection prematurely resets the input state. The soft label generator therefore uses a split-normal Gaussian centred on each onset frame: a wide left standard deviation and narrow right standard deviation for Make, and narrow left and wide right for Break. This encodes directional uncertainty directly into the training target, preventing the model from being equally penalised for both error directions.

**4.4.3 Fully Convolutional Network.** The dilated residual FCN operates directly on raw normalized sensor streams without manual feature engineering. Stacked dilated convolutions grow the receptive field exponentially: with eight dilation stages and a kernel size of 7, the effective receptive field spans approximately 3 060 frames (≈3 s at 1 kHz), allowing the model to condition each frame's prediction on long-range temporal context—pre-click muscle tension ramp-ups and post-click rebound dynamics that fall well outside any practical sliding window. The model is trained end-to-end on a three-channel Gaussian heatmap target, producing soft onset probability curves rather than hard class assignments. The trade-off is higher computational cost: the FCN requires a GPU or significantly longer CPU training time, is less interpretable than tree-based models, and benefits more strongly from larger datasets. It is therefore positioned as the high-accuracy architecture for offline post-processing, while the Random Forest handles latency-sensitive or resource-constrained deployment.

### 4.5 From Classification to Segmentation

The per-frame classifier produces a four-class probability array over the full recording. Converting these continuous probability outputs into discrete (start, end) click segments requires three sequential design choices: identifying candidate onset regions, electing a single representative frame from each region, and pairing each Make onset region with its corresponding Break onset region.

**Candidate run detection.** Two separate probability channels are extracted from the classifier output: the Make Onset probability and the Break Onset probability. Contiguous regions above a configurable threshold in each channel are identified as candidate runs. For Make runs, a Schmitt-trigger hysteresis is applied: a run opens when the Make probability exceeds the make threshold and remains open until it falls below half that threshold. This bridging behavior prevents a single physical press—whose probability hump can dip below the threshold due to quantization noise or softmax saturation—from being fragmented into multiple spurious candidates. Break runs are detected with a simpler thresholding scheme.

**Onset frame election.** Within each candidate run, a single frame must be elected as the representative onset. Three strategies are evaluated: **peak** selects the argmax frame; **plateau median** identifies the set of frames within 0.5% of the peak probability (the plateau) and returns the median frame index, falling back to the argmax when the plateau collapses to a single point; **steepest rise/fall** selects the frame of maximum gradient, corresponding to the inflection point of the probability curve and providing an estimate that is independent of plateau saturation effects. The optimal strategy and its associated make and break threshold values are selected jointly via a Bayesian hyperparameter sweep (Optuna) over the validation set, optimizing the Intersection over Union (IoU) between predicted and ground-truth click segments.

**Make–Break pairing.** Given a set of elected Make onset frames and Break onset frames, a final matching step assigns each Make to exactly one Break. Three pairing strategies are available. The simplest is a **greedy first-fit** approach: each Make is paired with the first eligible Break that falls within the Make run's search window. This serves as a baseline. The second strategy formulates the assignment as a **linear sum assignment problem** (Hungarian algorithm), constructing a cost matrix whose entry $(i, j)$ is the negative geometric mean of the Make and Break confidence scores; the global minimum-cost assignment is then solved exactly in $O(n^3)$ time, ensuring no Break is claimed by more than one Make. The third strategy extends the Hungarian approach with a **trained pairing scorer**: a logistic regression classifier, fit on positive and negative (Make, Break) pairs extracted from the training set, replaces the raw confidence product as the cost signal. The scorer uses six features per candidate pair—Make confidence, Break confidence, log-duration between the elected frames, Make run width, Break run width, and the number of competing Break runs within the Make's search window—and is trained with balanced class weights to handle the natural scarcity of positive (correct) pairs. The

optimal pairing strategy is selected by evaluating all three on the validation set IoU, and the chosen strategy is stored alongside the model weights so inference uses identical pairing logic.

### 4.6 Parameter Optimization

After click-intent boundaries are extracted, the optimizer estimates the hardware threshold pair that best reproduces those boundaries under firmware-faithful trigger simulation.

**4.6.1 Firmware State Machine Simulation.** Each candidate threshold pair is evaluated with a frame-accurate simulation of firmware trigger logic.

In **simple-threshold mode**, the simulator is a 2-state machine: press is emitted when the make signal crosses the make threshold upward, and release is emitted when the break signal crosses the break threshold downward.

In **Rapid Trigger mode**, the simulator is a 3-state machine (Idle → Pressed → RT-Released) with rolling extrema. After actuation, release is emitted when depth falls by at least RT sensitivity below the running peak. From RT-Released, re-actuation occurs when depth rises by at least RT sensitivity above the running valley. The state resets to Idle only when the signal returns to zero, matching firmware reset semantics.

To preserve firmware behavior while keeping search efficient, signals are quantized before evaluation and searched on a discrete, hardware-relevant threshold grid.

**4.6.2 Search Space and Objective.** Optimization is discrete and exhaustive over the candidate threshold grid, yielding deterministic and reproducible solutions at exposed hardware resolution.

For each candidate, simulated click intervals are matched to labeled intervals using **overlap-constrained monotonic matching**. A label is matched only if at least one simulated interval overlaps that label interval. If multiple simulated intervals overlap the same label (oscillation/chatter), they are collapsed into one canonical matched interval using earliest make and latest break.

Let

$$\begin{aligned} N &= \text{number of labeled clicks,} \\ TP &= \text{number of matched labels,} \\ FN &= N - TP, \\ FP &= \text{number of unmatched simulated intervals.} \end{aligned}$$

For matched pair $i$, define make and break boundary errors:

$$e_i^{(m)} = t_{i,\text{sim}}^{(m)} - t_{i,\text{label}}^{(m)},$$
$$e_i^{(b)} = t_{i,\text{sim}}^{(b)} - t_{i,\text{label}}^{(b)}.$$

Early errors are asymmetrically penalized with multiplier $\kappa \geq 1$:

$$\phi_\kappa(e) = |e|\,(1 + (\kappa - 1)\mathbf{1}[e < 0])\,.$$

Per-match timing distance is

$$d_i = w_{\text{make}}\,\phi_\kappa\left(e_i^{(m)}\right) + (1 - w_{\text{make}})\,\phi_\kappa\left(e_i^{(b)}\right),$$

where $w_{\text{make}} \in [0,1]$ controls make-vs-break priority.

Detection penalties are parameterized by base ms-equivalent detection cost $c_{\text{det}}$ and recall/precision bias $\beta$:

$$c_{\text{miss}} = c_{\text{det}} e^{\beta}, \qquad c_{\text{fp}} = c_{\text{det}} e^{-\beta}.$$

The final loss is

$$\mathscr{L} = \begin{cases} \infty, & TP = 0, \\ \dfrac{1}{TP}\displaystyle\sum_{i=1}^{TP} d_i + \dfrac{c_{\text{miss}} FN + c_{\text{fp}} FP}{N}, & TP > 0. \end{cases}$$

This yields interpretable controls: $w_{\text{make}}$ sets make-vs-break timing emphasis, $\kappa$ controls early-click severity, and $(c_{\text{det}}, \beta)$ control overall detection strictness and recall-precision preference.

**4.6.3 Pareto Frontier and Output.** For each feature-pair configuration, the optimizer returns all evaluated candidates, the minimum-loss operating point, and a Pareto frontier over {error rate, average click distance} for interactive inspection.

Detailed serialization choices (grid layout, sen-

tinels, and per-combination diagnostics) are implementation concerns and are described in Section 5.3.

# 5. Implementation

This chapter describes the software and hardware implementation of the system designed in Chapter 4.

## 5.1 Frontend and Telemetry Stack

The user-facing layer is built within GSense, providing the orchestration dashboard, recording states, and test minigames. Telemetry ingestion is managed primarily by GXT-Devio via HIDPP and seamlessly synchronized by the Nightfall background service. This layered composition ensures that 1 kHz hardware polling runs safely out-of-process, leaving the GSense frontend free to organize workflows without stalling the main analytical execution arrays.

## 5.2 Machine Learning Backend

The machine learning pipeline is encapsulated in a dedicated Python intent-detection backend built with scikit-learn, LightGBM, and PyTorch.

**5.2.1 Loading and Frame Extraction.** Each recording file is paired with a JSON label sidecar validated at load time. The 120-level analog depth stream is normalised to $[0,1]$ by dividing by 120, and binary digital press signals are cast to $\{0.0, 1.0\}$. Only segments annotated as `source=manual` are ingested; algorithmically generated segments are discarded. Each manually annotated click segment maps to three frame-level classes: Make Onset, Click Hold, and Break Onset. Onset zones extend up to $\pm 8$ frames from each boundary marker to absorb annotation imprecision, but are clamped to at most one-third of the segment length and against adjacent segment boundaries to prevent cross-segment label contamination. A mtime-keyed in-memory cache prevents redundant re-parsing of unchanged files.

**5.2.2 Preprocessing and Signal Augmentation.** The analog channels optionally receive temporal smoothing (moving average, Gaussian, Savitzky-Golay, median, or EMA). Kinematic derivative channels—velocity, acceleration, jerk, and their absolute-value variants—are computed via zero-phase Akima-spline upsampling ($2\times$) followed by centered finite differences and downsampling, eliminating causal shift. A resting-drift channel is derived as the difference of a fast EMA ($\tau \approx 20$ ms) and a slow EMA ($\tau \approx 200$ ms), both applied zero-phase. Continuous channels are optionally decimated using an 8th-order zero-phase Chebyshev IIR low-pass filter before striding; discrete digital channels are strided only. A rolling-window activity mask (window 25 frames, amplitude threshold 0.02, variance threshold $10^{-4}$) trims inactive interior gaps longer than 500 frames while retaining 50-frame margins on each side and all labeled frames, splitting the recording into independent sub-sequences.

**5.2.3 Splitting, Normalization, and Sample Weighting.** Sub-sequences are partitioned into train, validation, and test at the sequence level. The preferred strategy is participant-level k-fold cross-validation: sequences are grouped by `user_id` (extracted from the file path via `p\d+` regex), each fold holds out exactly one participant group, and sequences without a `user_id` always go to training. Z-score normalization is fitted on the training partition only. For kinematic channels (velocity, acceleration, jerk, drift) the standard deviation is computed over non-background frames exclusively to prevent the background mass from compressing the feature scale. Binary digital and constant per-recording configuration channels are excluded from normalization.

Click-balanced per-frame sample weights equalize participant contributions. For participant $u$ with $C_u$ click events and $b_u$ background frames, the weight of a frame belonging to click event $e$ spanning $|F_e|$ frames is $(1/C_u)/|F_e|$, and the weight of a background frame is $1/b_u$. Weights are then rescaled per class to preserve the click/background mass ratio, and capped at a maximum ratio of 10 via iterative convergence. The Random Forest additionally applies inverse-frequency class weights scaled by configurable per-class multipliers, providing an independent knob for the cost of each misclassification type.

**5.2.4 Feature Extraction and Model Ingestion.** For the Random Forest and Gradient Boosted Trees, sliding-window feature extraction produces one tabular vector per frame. Each window spans 33 frames (±16 around the centre frame, padded by edge-repetition at boundaries) and computes seven per-channel features by default: window mean, first-to-last slope, peak, velocity at the centre frame, time-to-peak, minimum, and time-to-trough. Five cross-channel features are always appended: L/R analog correlation, its absolute value, whether the left digital signal fires during an analog rise, the same for the right, and the maximum analog value across both sides. For the base four-channel input this yields $4 \times 7 + 5 = 33$ features per frame, growing by 7 for each additional derived channel enabled. An extended 13-feature bank (adding range, zero-crossing rate, TKEO energy, kurtosis, and waveform length) is available but disabled by default. Extraction is chunked in blocks of 50,000 frames to bound peak memory. Optional context probes append raw or locally averaged analog samples at configurable temporal offsets (e.g. ±50 ms, ±100 ms).

The FCN bypasses tabular extraction and receives raw $[C, T]$ tensors directly. Training labels are converted to a 3-channel heatmap: channels 0 and 1 carry Gaussian probability bumps ($\sigma = 8$ frames) centred on each Make and Break onset frame respectively, and channel 2 carries a binary hold mask. A focal binary cross-entropy loss is applied per channel per frame, with per-frame sample weights scaling the loss directly. When sequence-window training is used, an 8-frame zero-weight gap is inserted between recordings in the concatenated hypersequence so no sliding window can span a boundary.

### 5.3 Optimization Pipeline Integration

The parameter optimizer is implemented as a standalone Rust service that communicates with the front-end client via named-pipe IPC. It receives preprocessed analog signals and labeled click regions from the ML manager, performs an exhaustive discrete search over candidate thresholds, and returns both best-solution and analysis artifacts as JSON outputs.

The core of the service is a frame-accurate simulation of firmware trigger logic, illustrated in Figures 5.1 and 5.2. The deployed optimization path uses depth for both make and break channels, with candidate thresholds $(T_m, T_r)$ where $T_m$ is Actuation Point and $T_r$ is Rapid Trigger sensitivity. In this path, the simulator runs a 3-state FSM (Idle → Pressed → RT-Released):

- Idle → Pressed if $m_i \geq T_m$ (emit make, initialize running peak),
- Pressed → RT-Released if $b_i \leq \text{peak} - T_r$ (emit break, initialize running valley),
- RT-Released → Pressed if $b_i \geq \text{valley} + T_r$ (emit make),
- RT-Released → Idle only when the signal returns to zero ($b_i \leq 0$ or valley $\leq 0$), matching firmware reset behavior.

For non-depth break features, the backend also supports a simple 2-state threshold mode with candidate $(T_m, T_b)$:

- unpressed → pressed when $m_i \geq T_m$,
- pressed → unpressed when $b_i \leq T_b$.

Signals are quantized before evaluation so search remains firmware-faithful and efficient. In the deployed depth-depth path, thresholds are evaluated exhaustively over the exposed 10-level depth grid.

For each threshold candidate, simulated intervals are aligned to labels using overlap-constrained monotonic matching. The service computes timing and detection metrics (matched, missed, false positives, make-early, break-early, and latency summaries), evaluates the objective described in Section 4.6, and stores per-candidate simulated intervals and aggregate statistics.

Output artifacts include:

- the complete set of evaluated candidates,
- the minimum-loss solution,
- a Pareto frontier over {error rate, average click distance},
- landscape data for visualization.

For landscape visualization, the deployed

depth-depth path emits its exhaustive evaluated grid directly (10×10). Generalized feature-pair paths emit a fixed 40×40 binned grid, with unvisited cells marked by sentinel loss values.

### 5.4 Data Acquisition Campaign

The development and training of the machine learning backend required the construction of a robust ground-truth dataset spanning diverse ergonomic and biomechanical profiles. Data was collected from 69 participants via active field deployments at two major public gaming events (BIC and PolyLAN).

**5.4.1 Controlled Task Dataset (BIC Event).** The first deployment cohort ($N$ = 26) comprised a mix of highly ranked competitive gamers and casual novices playing *Counter-Strike 2* (CS2). The protocol ran for approximately 60 minutes per user and was strictly structured to isolate specific click dynamics:

- **Configuration A/B Testing:** Following a uniform warmup phase, users played two 15-minute game blocks. The device hardware profile was hot-swapped halfway through the session, alternating between an aggressive configuration (AP at 1, RT at 1) and a highly conservative configuration (AP at 5, zero Rapid Trigger). The presentation order was randomized to mitigate sequence bias.
- **Weapon Rotations:** Within each 15-minute block, users were instructed to rotate their active weapon every five minutes. The arsenal was specifically chosen to evoke distinct motor patterns: the *USPS* pistol for rapid individual tapping fatigue, the *M4A1* rifle for sustained tracking and spraying, and the *AWP* sniper rifle for slow, deliberate holding and release.

Qualitative user feedback regarding the switch feel was collected asynchronously after the sessions via standardized surveying.

**5.4.2 Free-Play Dataset (PolyLAN Event).** The secondary dataset ($N$ = 43) focused on unstructured free-play in a high-density LAN environment. Participants engaged in approximately 40-minute play sessions utilizing their preferred competitive title (*League of Legends*, *CS2*, or *Valorant*). Similar to the controlled dataset, the hardware configurations were silently swapped from conservative to aggressive baseline profiles near the 20-minute mark to capture a wider net of organic responses.

The aggregated dataset generated from these two campaigns ultimately produced hundreds of hours of raw 1 kHz telemetry, demanding the variance entropy scoring (detailed in Section 4) to isolate periods of genuine engagement before entering the classification pipelines.

**5.4.3 Annotated Dataset Summary.** Following manual annotation and variance-based trimming, the final curated dataset comprises 132 labeled recording files from 39 participants, totaling approximately 12,196 seconds (~3.4 hours) of verified active telemetry. Left-channel annotations account for 39,088 labeled click cycles and right-channel annotations for 6,645 cycles, reflecting the predominance of left-button interactions in competitive gaming. The dataset was partitioned into training, validation, and test splits performed at the participant level to enforce strict subject isolation.

## 6. Evaluation

This chapter evaluates the performance of the machine learning classifiers and the downstream hardware parameter optimizer.

### 6.1 Evaluation Setup

**6.1.1 Metrics.** The segmenting classifier's quality is primarily measured via per-class classification accuracy alongside the absolute temporal detection error, expressed as the average millisecond divergence from the ground truth intention timestamp. The parameter optimizer is evaluated on its success in minimizing the simulation's tripartite loss: maintaining peak precision and recall whilst simultaneously minimizing latency.

**6.1.2 Baselines.** The optimizer evaluates hardware configuration changes linearly against the fixed threshold baselines (e.g., default firmware behaviour). The underlying detection engine evaluates success compared to raw heuristics mapping logic.

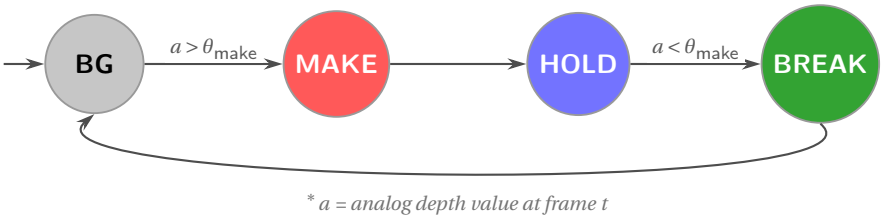


Figure 5.1: Simple threshold (no Rapid Trigger) FSM. A Make event fires when depth crosses above $\theta_{\text{make}}$; a Break event fires when depth falls back below the same threshold. The four states correspond directly to the four classifier output classes.

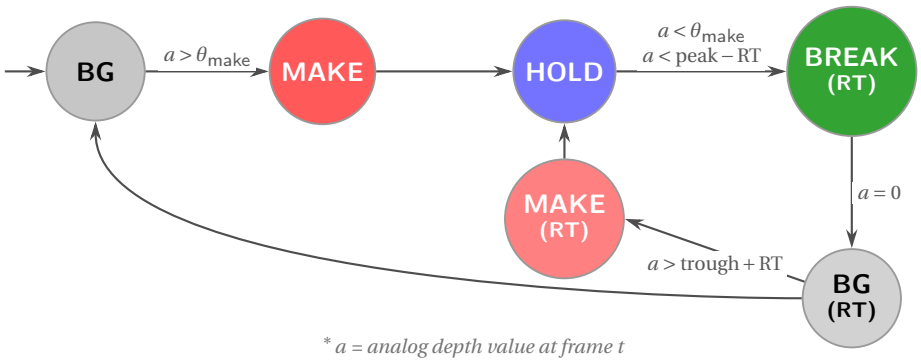


Figure 5.2: Rapid Trigger (RT) FSM. Once pressed, the button releases when depth drops by the RT sensitivity below the last observed peak. From the RT-released state the button re-actuates when depth rises above the last trough by the RT sensitivity, producing another Make onset; it fully resets to idle when depth returns to zero.

### 6.2 Click-Intent Classifier Results

Tasked with interpreting the analog stream, the Random Forest model driven by "distant context probes" achieved extraordinary performance levels. It successfully attained 97% segmentation accuracy on distinguishing genuine play intent from background data. Furthermore, the model provided extremely tight temporal bounding, showing merely a 7 ms average detection error on labeling constraints.

### 6.3 Ablation Studies

**6.3.1 Variance Entropy Scoring.** Considering that 80–90% of raw game session data operates purely as noisy background idleness, attempting to run models without first applying the variance-based entropy score trimming produces severe data imbalance, crippling model convergence and execution speeds.

**6.3.2 Contextual Probes.** Ablating the "distant context probes" verifies that analyzing isolated instant frames limits classification to naive static thresholding. Including pre- and post-frame analytical slices serves to contextualize the momentum of a single finger, allowing 97% classification precision.

### 6.4 Parameter Optimizer Results

The CMA-ES optimizer effectively simulated the hardware switch logic, validating the system's ability to hunt specific hardware tolerances to uniquely match physical telemetry. It successfully discovers configurations that improve objective measures by collapsing latency gaps within the parameters of strong recall and precision.

### 6.5 End-to-End User Validation Protocol (Project ARROW)

To close the loop from theoretical model accuracy to practical HCI outcomes, a structured end-to-end user validation protocol (Project ARROW) is designed. This evaluation assesses the subjective trust users place in the calibration recommendations and quantifies objective performance changes during actual gameplay.

**6.5.1 Study Objectives.** The user validation protocol operates on three primary objectives:

1. **Subjective Trust & Perception:** Assessing users' willingness to trust an autonomous calibration tool and accept its recommendation, specifically tracking feedback regarding how the "intent proficiency score" and recommendation UI are interpreted.
2. **Objective Performance (Zapper Trajectory):** Utilizing a custom standalone aiming minigame (Zapper) to analyze "stop-to-click" latency. A measurable reduction in this mechanical threshold latency verifies the optimizer's effectiveness over human-adjusted defaults.
3. **Misclick Monitoring (Performance Loss):** Actively logging any catastrophic regressions, explicitly tracking if a recommended configuration leads to unintended inputs or if the user actively rejects the calibration due to lack of trust.

(Placebo testing was intentionally excluded from the scope due to participant sample size limitations).

**6.5.2 Protocol Structure.** The 60-minute evaluation protocol is divided into four distinct phases:

- **Phase 1: Pre-Session Onboarding (0–5 mins).** Users define their base ergonomic settings (DPI). The mouse is strictly reset to a conservative standard configuration baseline. Initial "out-of-the-box" impressions are recorded and a participant ID is assigned.
- **Phase 2: Baseline Calibration (5–20 mins).** Utilizing a think-aloud protocol, users engage with the initial calibration wizard and play for 10–15 minutes while telemetry is recorded. This phase concludes with a 1-minute baseline performance round in the Zapper minigame.
- **Phase 3: Post-Optimization UX (20–40 mins).** Users review the UI presenting the optimization recommendation. If a user rejects the algorithm's configuration due to a lack of trust, it is recorded as a critical UX failure alongside their justification. Users then apply the optimization and play a secondary session, culminating in a final 1-

minute Zapper performance test to capture delta-latency improvements.
- **Phase 4: Offboarding & Validation (50–60 mins).** Technical verification ensures the GSense tracking history is correctly encapsulated for the specific participant ID, followed by final qualitative user surveying and debriefing.

### 6.6 Discussion

**6.6.1 Strengths.** The solution successfully operationalizes continuous hardware capability with machine learning inference, addressing the known difficulty of correctly tuning analog actuation devices manually. Identifying 1 kHz intent with 7 ms precision empowers the system to drastically override default performance blocks efficiently.

**6.6.2 User Trust and Adoption Roadblocks.** User evaluation provided a stark insight into adoption: users categorically distrust entirely autonomous hardware modification engines acting as black boxes. For production scale viability (such as integration into GHub and GHabits platforms), visual performance validations alongside objective data transparency (graphs outlining the recommended change) are structurally essential to foster adoption.

## 7. Conclusion

### 7.1 Summary

This thesis presented an offline adaptive calibration pipeline for analog peripherals with continuous depth sensing. The work addresses a practical limitation of current firmware: static actuation thresholds that do not adapt to per-user biomechanics, device variability, or within-session fatigue.

The main contributions are:

- A **hybrid labeling pipeline** that derives initial frame-level `MAKE_ONSET`, `CLICK_HOLD`, and `BREAK_ONSET` annotations from algorithmic criteria (depth geometry, speed, and acceleration) and then refines boundary quality through model-assisted iteration. A geometric weak-supervision heuristic is combined with asymmetric soft labels to model directional timing uncertainty at boundaries.
- A **four-class temporal sequence classifier** with two model paths. The shallow path (Random Forest, Gradient Boosted Trees) operates on hand-crafted sliding-window features and supports rapid per-session retraining. The deep path (dilated residual FCN) operates on raw four-channel telemetry with a learned $\pm 1530$ ms receptive field, trading higher context for higher compute cost.
- A **joint CMA-ES optimizer** that simulates the peripheral firmware's Rapid Trigger state machine frame-by-frame and finds Pareto-optimal (AP, RT) configurations under latency and false-positive objectives, replacing manual threshold sweeps with a data-driven trade-off analysis.

In aggregate, these components provide an end-to-end workflow in which a user recording can be automatically labeled, modeled, and optimized to produce evidence-based AP/RT recommendations with minimal manual intervention.

### 7.2 Limitations

Several limitations constrain the generality of the current system:

- **Small evaluation cohort.** The system was evaluated on a limited number of users and sessions. Cross-user generalization—whether a model trained on one user's data transfers meaningfully to another—was not demonstrated at scale. The per-session training approach mitigates this by avoiding cross-user transfer entirely, but the cold-start problem for first-time users with no prior recordings is not addressed.
- **Auto-labeler timing errors.** The geometric labeling heuristic introduces systematic timing bias of up to $W_b$ frames. While asymmetric soft labels partially absorb this uncertainty, they do not eliminate it. Make-onset F1 is sensitive to the plateau patience parameter, confirming that labeler quality is a first-order determinant of classification performance.
- **Offline-only inference.** The OfflineFCN's symmetric non-causal padding is incompatible with streaming inference on the peripheral MCU. Any deployment on the on-device path would require a causal (left-padded) architectural variant,

which reduces the per-layer receptive field contribution by half.

- **Drift channel unused in the optimizer.** The drift channel ($EMA_{20} - EMA_{200}$) is computed and exposed in the session review UI but is not integrated into the optimizer's objective function. As a result, the system does not currently adapt AP/RT recommendations in response to detected fatigue-induced resting-weight shift.

### 7.3 Future Work

1. **On-device deployment.** Quantize the Offline-FCN to INT8 and evaluate inference latency on the peripheral RISC-V MCU. Develop a causal (left-padded) variant compatible with streaming inference. A causal model would convert the offline classifier into a real-time intent detector, enabling the pre-report filtering described in the Background.
2. **Multi-session accumulation.** Train on pooled data from a user's full recording history rather than re-training from scratch each session. A multi-session model accumulates evidence about the user's long-term biomechanical signature, reducing label noise from individual sessions and improving performance on edge cases (fatigue-modulated dynamics, device warm-up transients).
3. **Drift-aware re-calibration.** Integrate the drift channel into the optimizer's objective: trigger an automatic re-calibration recommendation when sustained positive drift exceeds a per-user threshold, indicating flexor-fatigue onset. This closes the loop between the fatigue diagnostic described in Section **??** and the AP/RT adaptation..
4. **Cross-user pre-training.** Pre-train the FCN on pooled data from multiple users and fine-tune per-session, reducing the cold-start problem for new users with short recording histories. A pre-trained backbone should require fewer labeled frames to reach stable performance than a randomly initialized model.
5. **Product and UX integration.** Integrate the recommendation workflow into the UX Exploration v2 flow so users can inspect why a suggested AP/RT point is selected (trade-off plots, confidence signals, and comparable alternatives), then harden the same interface and telemetry contracts for downstream integration into GHub/GHabits.

# Bibliography


[1] Hiroshi Akima. “A New Method of Interpolation and Smooth Curve Fitting Based on Local Procedures”. In: *Journal of the ACM* 17.4 (1970), pp. 589–602. DOI: 10.1145/321607.321609.

[2] C.W. Antuvan and L. Masia. “An LDA-Based Approach for Real-Time Simultaneous Classification of Movements Using Surface Electromyography”. In: *IEEE Transactions on Neural Systems and Rehabilitation Engineering* (2019).

[3] Attack Shark. *Tap-Hold Functionality: Doubling Macro Space via Advanced Firmware and Hall Effect Rapid Trigger Advantage*. Hardware Engineering & Firmware Blogs. 2026.

[4] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling”. In: *arXiv preprint arXiv:1803.01271* (2018).

[5] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[6] Y. Chen et al. “Human-Piloted Drone Racing: Visual Processing, Control, and the Impact of Rapid Trigger Keyboards”. In: *IEEE Transactions on Human-Machine Systems* (2024).

[7] Joanne DiFrancisco-Donoghue, Jerry Balentine, George Schmidt, and Hallie Zwibel. “Managing the health of the eSport athlete: an integrated health management model”. In: *BMJ Open Sport & Exercise Medicine* (2019). DOI: 10.1136/bmjsem-2018-000467.

[8] Garrick N. Forman, Lucas P. Melchiorre, and Michael W. R. Holmes. “Impact of repetitive mouse clicking on forearm muscle fatigue and mouse aiming performance”. In: *Applied Ergonomics* (2024). Focuses on biomechanical degradation and clicking-induced aiming inaccuracy. DOI: 10.1016/j.apergo.2024.104284.

[9] J. Fox. *The Logitech Superstrike has allowed me to get excited over some genuinely new PC gaming technology*. PC Gamer. Accessed: 2026-03-19. 2026.

[10] Jerome H. Friedman. “Greedy Function Approximation: A Gradient Boosting Machine”. In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. DOI: 10.1214/aos/1013203451.

[11] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. arXiv preprint arXiv:1604.00772. 2016. eprint: arXiv:1604.00772.

[12] C. Heimhofer, S. Koblitz, M. Bächinger, and N. Wenderoth. “Finger-specific representations are sharpened during a fatiguing motor task”. In: *Current Issues in Sport Science (CISS)* (2024). Analyzes neuromotor compensation and brain representation during finger fatigue. DOI: 10.36950/2024.2ciss048.

[13] J. Hernandez, P. Paredes, A. Roseway, and M. Czerwinski. “Under Pressure: Sensing Stress of Computer Users”. In: *Proceedings of the 2014 ACM CHI Conference on Human Factors in Computing Systems*. 2014.

[14] M. Hwu. *Mouse And Keyboard Ergonomics Tips From A Physical Therapist*. 1-HP Esports Health. 2024.

[15] C. Imianosky, D. A. Santos, and L. Dilillo. “Efficient TinyML Inference on a Fault-Tolerant RISC-V SoC with Vector Extension”. In: *Proceedings of the IEEE*. 2025.

[16] J.H. Kim and P.W. Johnson. “Fatigue development in the finger flexor muscle differs between keyboard and mouse use”. In: *European Journal of Applied Physiology* (2014).

[17] S. Kim, B. Lee, and A. Oulasvirta. “Impact Activation Improves Rapid Button Pressing”. In: *Proceedings of the 2018 ACM CHI Conference on Human Factors in Computing Systems*. 2018.

[18] Logitech. *Industry's First Gaming Mouse Featuring Logitech G's Revolutionary SUPERSTRIKE Technology*. https://www.logitech.com/blog/2025/09/17/industrys-first-gaming-mouse-featuring-logitech-gs-revolutionary-superstrike-technology/. Accessed: 2026-03-19. 2026.

[19] Logitech Europe S.A. "Hybrid switch for an input device ". US20230022831A1. Patent covering the Haptic Inductive Trigger System (HITS) mechanism. 2025.

[20] I.S. MacKenzie and A. Oniszczak. "A Comparison of Three Selection Techniques for Touchpads". In: *Proceedings of the 1998 ACM CHI Conference on Human Factors in Computing Systems*. 1998.

[21] L. de-Marcos, J.-J. Martínez-Herráiz, J. Junquera-Sánchez, C. Cilleruelo, and C. Pages-Arévalo. "Comparing Machine Learning Classifiers for Continuous Authentication on Mobile Devices by Keystroke Dynamics". In: *MDPI Electronics* (2021).

[22] C. McGee and K. Ho. "Tendinopathies in Video Gaming and Esports". In: *Frontiers in Sports and Active Living* (2021).

[23] John V. Monaco. "SoK: Keylogging Side Channels". In: *IEEE Symposium on Security and Privacy (S&P)*. 2018, pp. 211–228. DOI: 10.1109/SP.2018.00026.

[24] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, and Koray Kavukcuoglu. "WaveNet: A Generative Model for Raw Audio". In: *arXiv preprint arXiv:1609.03499* (2016).

[25] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. "Snorkel: Rapid Training Data Creation with Weak Supervision". In: *The VLDB Journal* 29.2–3 (2020), pp. 709–730. DOI: 10.1007/s00778-019-00552-1.

[26] Jacob Ridley. *Best Hall effect keyboards in 2026: the fastest, most customizable keyboards for competitive gaming*. PC Gamer. 2026.

[27] Abraham Savitzky and Marcel J. E. Golay. "Smoothing and Differentiation of Data by Simplified Least Squares Procedures". In: *Analytical Chemistry* 36.8 (1964), pp. 1627–1639. DOI: 10.1021/ac60214a047.

[28] Raghubir Singh and Sukhpal Singh Gill. "Edge AI: A survey". In: *Internet of Things and Cyber-Physical Systems* (2023). DOI: 10.1016/j.iotcps.2023.02.004.

[29] G. Stragapede, P. Delgado-Santos, R. Tolosana, R. Vera-Rodriguez, R. Guest, and A. Morales. "TypeFormer: Transformers for Mobile Keystroke Biometrics". In: *arXiv preprint arXiv:2212.13075 / Neural Computing and Applications* (2022).

[30] TechRadar. *Logitech G Pro X2 Superstrike Review: Haptic Inductive Trigger System Analysis*. https://www.techradar.com/computing/mice/logitech-g-pro-x2-superstrike-review. 2026.

[31] C. Tholl, L. Hansen, and I. Froböse. "Wrist extensor fatigue and game-genre-specific kinematic changes in esports athletes: a quasi-experimental study". In: *Journal of Electronic Gaming and Esports* (2025). PMCID: PMC12400618. Compares high-APM genres (FPS vs MOBA) and muscular recovery.

[32] C. Tholl, L. Hansen, and I. Froböse. "Wrist extensor fatigue and game-genre-specific kinematic changes in esports athletes: a quasi-experimental study". In: *Journal of Electronic Gaming and Esports* (2025). PMCID: PMC12400618.

[33] S. Trewin. "Automating Accessibility: The Dynamic Keyboard". In: *Proceedings of the 6th International ACM SIGACCESS Conference on Computers and Accessibility (Assets)*. 2003.

[34] Author Unknown. "Fine-Tuning Magnetic Sensors for Dynamic Movement Stops". In: *Hardware Engineering* (Unknown).

[35] R. Wang, X. Zhu, A. Chen, J. Xu, L. Winterbottom, D. Nilsen, J. Stein, and M. Ciocarlie. "ReactEMG: Stable, Low-Latency Intent Detection from sEMG via Masked Modeling". In: *arXiv preprint arXiv:2506.19815* (2025).

[36] Wooting Technologies. *Rapid Trigger: A Must for Games, Here is Why*. `https://wooting.io/rapid-trigger`. Accessed: 2026-03-19. 2022.